

BASES DE DATOS II · INGENIERÍA DE SOFTWARE · CURSO 2026–2027

Restricciones de integridad del diseño

Sistema de gestión para la generación automática de horarios docentes

Documento base: Requerimientos informacionales (RI-01 ... RI-21, RI-R1 ... RI-R8) y Requerimientos funcionales

Motor objetivo: PostgreSQL (modelo relacional) · ClickHouse (histórico analítico)

Versión: 1.0 · octubre de 2026

Elaborado por: Equipo de requerimientos

Contenido

1. Propósito, alcance y convenciones	
2. Esquema relacional de referencia	
3. Restricciones de dominio (RD)	
4. Restricciones de tupla (RT)	
5. Restricciones de clave y unicidad (RC)	
6. Restricciones de integridad referencial (RR)	
7. Restricciones de cardinalidad y participación (RP)	
8. Restricciones de especialización (RE)	
9. Restricciones semánticas (RS)	
10. Restricciones dinámicas o de transición (RX)	
11. Restricciones del histórico analítico (RH)	
12. Reglas que <i>no</i> son restricciones de integridad	
13. Implementación en PostgreSQL	
14. Inconsistencias detectadas y decisiones pendientes	

1. Propósito, alcance y convenciones

Este documento define las **restricciones de integridad** que debe satisfacer todo estado válido de la base de datos del sistema. Se derivan de los requerimientos informacionales y de sus reglas de información, del documento de requerimientos funcionales y del enunciado del proyecto. Cada restricción indica su origen y el mecanismo con que se garantiza en PostgreSQL.

Las restricciones son condiciones sobre los **datos**. Las reglas que dependen de **quién** ejecuta una operación (permisos, visibilidad) se tratan aparte en la sección 12, porque no se expresan sobre el estado de la base de datos.

1.1 Clasificación y codificación

Prefijo	Tipo	Qué controla
RD	Dominio	Valores admisibles de un atributo aislado.
RT	Tupla	Relación entre atributos de una misma fila.
RC	Clave y unicidad	Identificación de las tuplas (claves primarias y candidatas).
RR	Referencial	Existencia de la tupla referenciada (claves foráneas) y acción ante borrado o actualización.

Prefijo	Tipo	Qué controla
RP	Cardinalidad y participación	Mínimos y máximos de las interrelaciones del MER.
RE	Especialización	Cobertura y solapamiento de la jerarquía de usuarios.
RS	Semántica (estática)	Reglas del negocio que involucran varias tuplas o relaciones.
RX	Dinámica (transición)	Cambios de estado permitidos y atributos inmutables.
RH	Histórico	Consistencia entre PostgreSQL y el almacén analítico ClickHouse.

1.2 Origen de cada restricción

- RI-xx** Se deduce directamente de los requerimientos informacionales o de sus reglas (RI-R1 ... RI-R8).
- RF** Se deduce de los requerimientos funcionales o del enunciado.
- Supuesto** Es una decisión de diseño del equipo que **aún no está validada** por los profesores. Hay que confirmarla o descartarla (ver sección 14).

1.3 Notación

Las restricciones complejas se formalizan en lógica de primer orden sobre las relaciones del esquema. Se usa $t.A$ para el valor del atributo A en la tupla t, y $\Rightarrow$, $\wedge$, $\vee$, $\neg$, $\forall$, $\exists$ con su significado habitual. En el texto se usan los nombres de los requerimientos (camelCase, por ejemplo `idHorario`). En el SQL se usa snake_case (`id_horario`), que es la convención de PostgreSQL y de EF Core con `UseSnakeCaseNamingConvention`.

Limitación: el diagrama entidad-relación (MER) no se recibió junto a este encargo: el enlace compartido apunta a la página de inicio de Excalidraw, no al diagrama. Las cardinalidades se tomaron de la redacción de los requerimientos informacionales (por ejemplo, “incluye una o más asignaturas”). Si el MER las representa de otra forma, hay que revisar la sección 7.

2. Esquema relacional de referencia

Las restricciones se expresan sobre el siguiente esquema, obtenido al transformar las entidades RI-01 ... RI-19. Las claves primarias van subrayadas y las foráneas se marcan con $\rightarrow$. Las relaciones **ProgramaAsignatura**, **Matricula**, **BloquePrograma** y **HorarioBloque** salen de las interrelaciones N:M descritas en los requerimientos.

Relación	Atributos	Origen
Usuario	<u>idUsuario</u> , nombre	RI-01
Administrador	<u>idUsuario</u> $\rightarrow$ Usuario	RI-01
JefeDepartamento	<u>idUsuario</u> $\rightarrow$ Usuario	RI-01
Coordinador	<u>idUsuario</u> $\rightarrow$ Usuario, especialidad	RI-03
Estudiante	<u>idUsuario</u> $\rightarrow$ Usuario, edad, idGrupo $\rightarrow$ Grupo	RI-02
Grupo	<u>idGrupo</u> , nombre	RI-07
ProgramaAcademico	<u>idPrograma</u> , nombre, planEstudio	RI-05

Relación	Atributos	Origen
Asignatura	<u>idAsignatura</u> , nombre	RI-06
ProgramaAsignatura	<u>idPrograma</u> → ProgramaAcademico, <u>idAsignatura</u> → Asignatura	RI-05
Matricula	<u>idEstudiante</u> → Estudiante, <u>idPrograma</u> → ProgramaAcademico	RI-02
CoordinadorPrograma	<u>idUsuario</u> → Coordinador, <u>idPrograma</u> → ProgramaAcademico, puedeGenerar, puedeValidar	RI-04
BloqueContenido	<u>idBloque</u> , descripcion, tipoActividad, nivelComplejidad, idCreador → Coordinador, idAsignatura → Asignatura	RI-08
BloquePrograma	<u>idBloque</u> → BloqueContenido, <u>idPrograma</u> → ProgramaAcademico	RI-08
ParametrosGeneracion	<u>idParametro</u> , proporcionTeorica, proporcionPractica, proporcionLaboratorio, nivelComplejidad	RI-09
PeriodoAcademico	<u>idPeriodo</u> , nombre, estado, fechaInicio, fechaFin	RI-11
Horario	<u>idHorario</u> , fechaCreacion, estado, (idCoordinador, idPrograma) → CoordinadorPrograma, idPeriodo → PeriodoAcademico, idParametro → ParametrosGeneracion, idRevisionOrigen → Revision (nulo)	RI-10
HorarioBloque	<u>idHorario</u> → Horario, <u>idBloque</u> → BloqueContenido, idPrograma	RI-10
Edificio	<u>idEdificio</u> , nombre, direccion	RI-14
Aula	<u>idAula</u> , nombre, capacidad, estado, idEdificio → Edificio	RI-13
Sesion	<u>idSesion</u> , fecha, horaInicio, horaFin, estado, (idHorario, idBloque) → HorarioBloque, idAula → Aula	RI-12
AsignacionHorario	<u>idEstudiante</u> → Estudiante, <u>idHorario</u> → Horario, idPrograma, idPeriodo, estado, fechaAsignacion	RI-15
Asistencia	<u>idEstudiante</u> → Estudiante, <u>idSesion</u> → Sesion, estadoAsistencia, fechaRegistro, observaciones	RI-16
Validacion	<u>idValidacion</u> , fechaValidacion, decision, observaciones, (idCoordinador, idPrograma) → CoordinadorPrograma, idHorario → Horario	RI-17
Revision	<u>idRevision</u> , fechaRevision, decision, observaciones, criteriosNuevaGeneracion, idJefe → JefeDepartamento, idHorario → Horario	RI-18
SolicitudCambio	<u>idSolicitud</u> , fechaSolicitud, motivo, estado, fechaResolucion, resultado, idEstudiante → Estudiante, idHorarioOrigen → Horario, idHorarioDestino → Horario (nulo), idCoordinadorApoyo → Coordinador (nulo), idCoordinadorResuelve → Coordinador (nulo)	RI-19

Atributos redundantes a propósito. HorarioBloque.idPrograma y AsignacionHorario.idPrograma / idPeriodo repiten datos que ya están en Horario. Se incluyen para que las restricciones RS-02, RS-04 y RS-09 se puedan garantizar con **claves foráneas compuestas e índices únicos**, sin triggers. Su consistencia con Horario queda asegurada por la propia FK compuesta (sección 13.2).

3. Restricciones de dominio (RD)

Código	Atributo(s)	Dominio	Origen
RD-01	Todos los <code>id...</code>	Entero positivo generado por el sistema (<code>GENERATED ALWAYS AS IDENTITY</code>).	Supuesto
RD-02	Usuario.nombre, Grupo.nombre, Asignatura.nombre, ProgramaAcademico.nombre, Edificio.nombre, Aula.nombre, PeriodoAcademico.nombre	Texto obligatorio, no vacío tras eliminar espacios, longitud máxima 150.	RI-01...RI-14
RD-03	Estudiante.edad	Entero entre 16 y 99.	RI-02 · Supuesto (rango)
RD-04	Coordinador.especialidad, ProgramaAcademico.planEstudio	Texto obligatorio no vacío.	RI-03, RI-05
RD-05	CoordinadorPrograma.puedeGenerar, puedeValidar	Booleano obligatorio, valor por defecto <code>false</code> .	RI-04
RD-06	BloqueContenido.tipoActividad	{TEORICA, PRACTICA, LABORATORIO}	RI-08
RD-07	BloqueContenido.nivelComplejidad, ParametrosGeneracion.nivelComplejidad	{BAJO, MEDIO, ALTO}	RI-08, RI-09
RD-08	BloqueContenido.descripcion	Texto obligatorio, no vacío.	RI-08
RD-09	ParametrosGeneracion.proporcion*	Entero entre 0 y 100 (porcentaje). Ver la nota de RT-01.	RI-09 · Supuesto (escala)
RD-10	Horario.estado	{GENERADO, VALIDADO, APROBADO, RECHAZADO}	RI-10 · Supuesto (valores)
RD-11	PeriodoAcademico.estado	{PLANIFICADO, ACTIVO, CERRADO}	RI-11 · Supuesto
RD-12	Sesion.estado	{PROGRAMADA, IMPARTIDA, CANCELADA}	RI-12 · Supuesto
RD-13	Aula.estado	{DISPONIBLE, MANTENIMIENTO, INACTIVA}	RI-13 · Supuesto
RD-14	Aula.capacidad	Entero > 0.	RI-13
RD-15	AsignacionHorario.estado	{ACTIVA, ANULADA}	RI-15 · Supuesto
RD-16	Asistencia.estadoAsistencia	{PRESENTE, AUSENTE, JUSTIFICADO}	RI-16 · Supuesto
RD-17	Validacion.decision	{VALIDADO, NO_VALIDADO}	RI-17 · Supuesto

Código	Atributo(s)	Dominio	Origen
RD-18	Revision.decision	{ APROBADO, RECHAZADO }	RI-18, RF
RD-19	SolicitudCambio.estado	{ PENDIENTE, EN_ATENCION, RESUELTA }	RI-19 · Supuesto
RD-20	SolicitudCambio.resultado	{ ACEPTADA, RECHAZADA } o nulo.	RI-19, RF
RD-21	SolicitudCambio.motivo	Texto obligatorio, no vacío.	RF
RD-22	Fechas de registro (fechaCreacion, fechaAsignacion, fechaRegistro, fechaValidacion, fechaRevision, fechaSolicitud)	Marca de tiempo con zona horaria (timestamptz), obligatoria, por defecto el instante de inserción y no posterior a él.	RI-10...RI-19
RD-23	Sesion.fecha / horaInicio / horaFin; PeriodoAcademico.fechaInicio / fechaFin	date y time , obligatorios.	RI-11, RI-12

4. Restricciones de tupla (RT)

Código	Restricción	Formalización	Origen
RT-01	Las tres proporciones de los parámetros suman exactamente 100 %.	p.proporcionTeorica + p.proporcionPractica + p.proporcionLaboratorio = 100	RI-09
RT-02	Un período empieza antes de terminar.	p.fechaInicio < p.fechaFin	RI-11
RT-03	Una sesión empieza antes de terminar.	s.horaInicio < s.horaFin	RI-12
RT-04	Una autorización de coordinador concede al menos un permiso; si no, la fila no tiene sentido.	cp.puedeGenerar v cp.puedeValidar	RI-04 · Supuesto
RT-05	Una solicitud está RESUELTA si y solo si tiene resultado, fecha de resolución y coordinador que la resolvió.	sc.estado = RESUELTA $\Leftrightarrow$ (sc.resultado $\neq$ nulo $\wedge$ sc.fechaResolucion $\neq$ nulo $\wedge$ sc.idCoordinadorResuelve $\neq$ nulo)	RI-19, RI-R8
RT-06	Una solicitud aceptada indica a qué horario se cambió, y ese horario es distinto del de origen.	sc.resultado = ACEPTADA $\Rightarrow$ (sc.idHorarioDestino $\neq$ nulo $\wedge$ sc.idHorarioDestino $\neq$ sc.idHorarioOrigen)	RI-19, RI-R8
RT-07	Una solicitud no se resuelve antes de presentarse.	sc.fechaResolucion $\neq$ nulo $\Rightarrow$ sc.fechaResolucion $\geq$ sc.fechaSolicitud	RI-19
RT-08	Un rechazo del jefe de departamento debe indicar los criterios para la nueva generación.	r.decision = RECHAZADO $\Rightarrow$ r.criteriosNuevaGeneracion $\neq$ nulo	RI-18, RF

Código	Restricción	Formalización	Origen
RT-09	Una validación negativa debe tener observaciones que la justifiquen.	$v.decision = NO_VALIDADO \Rightarrow v.observaciones \neq \text{nulo}$	Supuesto

Sobre RD-09 y RT-01. Si las proporciones se guardan como decimales (por ejemplo `numeric(4,3)`), un reparto de un tercio para cada tipo da $0,333 \times 3 = 0,999$, y la restricción “suma = 1” rechaza un caso válido. Con porcentajes enteros ($33 + 33 + 34 = 100$) la comprobación es exacta.

5. Restricciones de clave y unicidad (RC)

Código	Relación	Clave	Origen
RC-01	Todas	Toda relación tiene clave primaria; ningún atributo de la clave primaria admite nulos (integridad de entidad).	Modelo relacional
RC-02	Usuario y subtipos	PK <code>idUsuario</code> . En cada subtipo, la PK es también FK a Usuario (RE-01).	RI-01
RC-03	ProgramaAcademico	PK <code>idPrograma</code> ; clave candidata (<code>nombre</code> , <code>planEstudio</code>).	RI-05 · Supuesto
RC-04	Asignatura, Grupo, Edificio	PK <code>id...</code> ; clave candidata <code>nombre</code> .	Supuesto
RC-05	Aula	PK <code>idAula</code> ; clave candidata (<code>idEdificio</code> , <code>nombre</code>): dos aulas del mismo edificio no se llaman igual.	RI-13 · Supuesto
RC-06	PeriodoAcademico	PK <code>idPeriodo</code> ; clave candidata <code>nombre</code> .	Supuesto
RC-07	ProgramaAsignatura, Matricula, BloquePrograma, CoordinadorPrograma	PK compuesta por las dos FK. Así se impide repetir el mismo par.	RI-02, RI-04, RI-05, RI-08
RC-08	BloqueContenido	No pueden existir bloques duplicados. Clave candidata: (<code>idAsignatura</code> , <code>tipoActividad</code> , <code>descripción normalizada</code>). La descripción se normaliza pasando a minúsculas, sin espacios al inicio ni al final y con los espacios internos reducidos a uno.	RI-R1 · Supuesto (criterio)
RC-09	HorarioBloque	PK (<code>idHorario</code> , <code>idBloque</code>): un bloque aparece una sola vez en cada horario. Sus repeticiones semanales se representan como sesiones.	RI-10
RC-10	Horario	PK <code>idHorario</code> ; además se declaran <code>UNIQUE (idHorario, idPrograma)</code> y <code>UNIQUE (idHorario, idPrograma, idPeriodo)</code> como destino de las FK compuestas (sección 13.2).	Técnico
RC-11	AsignacionHorario	PK (<code>idEstudiante</code> , <code>idHorario</code>).	RI-15
RC-12	Asistencia	PK (<code>idEstudiante</code> , <code>idSesion</code>): un único registro de asistencia por estudiante y sesión.	RI-16

Código	Relación	Clave	Origen
RC-13	Validacion	PK idValidacion; UNIQUE (idHorario, idCoordinador) : un coordinador valida un horario una sola vez.	RI-17 · Supuesto
RC-14	Revision	PK idRevision; como máximo una revisión por horario, porque las dos decisiones posibles son finales (RX-01).	RI-18 · Supuesto
RC-15	SolicitudCambio	PK idSolicitud; como máximo una solicitud no resuelta por estudiante y horario de origen (índice único parcial).	Supuesto

6. Restricciones de integridad referencial (RR)

Por defecto toda FK es **obligatoria** (NOT NULL) y se declara con ON DELETE RESTRICT y ON UPDATE RESTRICT. El sistema guarda historial académico (horarios, asistencias, revisiones) que no debe desaparecer en cascada. Para dar de baja usuarios, aulas o períodos se usa **baja lógica** mediante el atributo estado o activo. Las excepciones a esta regla son las de la tabla.

Código	FK	Nulo	ON DELETE	Justificación
RR-01	Subtipos de Usuario → Usuario	No	CASCADE	El subtipo no existe sin su supertipo (RE-01).
RR-02	Estudiante.idGrupo → Grupo	No	RESTRICT	“Pertenece a un grupo” (RI-02).
RR-03	ProgramaAsignatura → ProgramaAcademico / Asignatura	No	CASCADE / RESTRICT	Tabla de asociación del programa: si se borra el programa, se borran sus filas. Una asignatura en uso no se puede borrar.
RR-04	Matricula → Estudiante / ProgramaAcademico	No	RESTRICT	Una matrícula referenciada por asignaciones no puede desaparecer (RS-09).
RR-05	CoordinadorPrograma → Coordinador / ProgramaAcademico	No	RESTRICT	Horarios y validaciones la referencian.
RR-06	BloqueContenido.idCreador → Coordinador; .idAsignatura → Asignatura	No	RESTRICT	“Lo crea un coordinador, aborda una asignatura” (RI-08). La consulta 6 necesita saber el creador.
RR-07	BloquePrograma → BloqueContenido / ProgramaAcademico	No	RESTRICT	La referencia HorarioBloque (RS-04).
RR-08	Horario.(idCoordinador, idPrograma) → CoordinadorPrograma	No	RESTRICT	“Lo genera un coordinador en programa” (RI-10). La FK es compuesta, así que el programa del horario es uno de los programas del coordinador.
RR-09	Horario.idPeriodo → PeriodoAcademico; .idParametro → ParametrosGeneracion	No	RESTRICT	RI-10 y RI-R6.

Código	FK	Nulo	ON DELETE	Justificación
RR-10	Horario.idRevisionOrigen → Revision	Sí	RESTRICT	Solo la tienen los horarios generados a raíz de un rechazo (RI-18, RS-12).
RR-11	HorarioBloque.(idHorario, idPrograma) → Horario; .(idBloque, idPrograma) → BloquePrograma	No	CASCADE / RESTRICT	FK compuestas que garantizan RI-R4 (RS-04).
RR-12	Aula.idEdificio → Edificio	No	RESTRICT	RI-13.
RR-13	Sesion.(idHorario, idBloque) → HorarioBloque; .idAula → Aula	No	RESTRICT	Una sesión solo imparte un bloque que forma parte de su horario (RI-12).
RR-14	AsignacionHorario.(idHorario, idPrograma, idPeriodo) → Horario; .(idEstudiante, idPrograma) → Matricula	No	RESTRICT	Garantizan RS-02 y RS-09.
RR-15	Asistencia.idEstudiante → Estudiante; .idSesion → Sesion	No	RESTRICT	RI-16. La asistencia es historial.
RR-16	Validacion.(idCoordinador, idPrograma) → CoordinadorPrograma; .idHorario → Horario	No	RESTRICT	“La realiza un coordinador en programa sobre un horario” (RI-17).
RR-17	Revision.idJefe → JefeDepartamento; .idHorario → Horario	No	RESTRICT	RI-18.
RR-18	SolicitudCambio.idEstudiante → Estudiante; .idHorarioOrigen → Horario	No	RESTRICT	RI-19.
RR-19	SolicitudCambio.idHorarioDestino → Horario; .idCoordinadorApoyo, .idCoordinadorResuelve → Coordinador	Sí	RESTRICT	Opcionales según RI-19. Se rellenan al resolver la solicitud (RT-05, RT-06).

7. Restricciones de cardinalidad y participación (RP)

Una FK **NOT NULL** garantiza el **mínimo 1 del lado N**. En cambio, el mínimo 1 del **lado 1 hacia el lado N** (“un programa incluye *una o más* asignaturas”) no se puede expresar con una FK. Para esos casos se usan **triggers de restricción diferidos**, que se comprueban al confirmar la transacción (sección 13.4).

Código	Interrelación	Cardinalidad	Mecanismo	Origen
RP-01	Estudiante pertenece a exactamente un Grupo.	N : 1 (total)	FK NOT NULL	RI-02
RP-02	Estudiante está matriculado en uno o más programas.	N : M, mín. 1	Trigger diferido	RI-02
RP-03	Programa incluye una o más asignaturas.	N : M, mín. 1	Trigger diferido	RI-05
RP-04	Bloque es creado por exactamente un coordinador y aborda exactamente una asignatura.	N : 1 (total)	FK NOT NULL	RI-08
RP-05	Bloque se utiliza en uno o más programas.	N : M, mín. 1	Trigger diferido	RI-08
RP-06	Horario es generado por exactamente un coordinador en programa, en un período y con unos parámetros.	N : 1 (total)	FK NOT NULL	RI-10

Código	Interrelación	Cardinalidad	Mecanismo	Origen
RP-07	Horario se genera a partir de uno o más bloques.	N : M, mín. 1	Trigger diferido	RI-10
RP-08	Sesión pertenece a un horario, imparte un bloque y ocurre en un aula.	N : 1 (total)	FK NOT NULL	RI-12
RP-09	Aula pertenece a exactamente un edificio.	N : 1 (total)	FK NOT NULL	RI-13
RP-10	Revisión la realiza exactamente un jefe sobre exactamente un horario, y puede originar cero o más horarios.	N : 1 / 1 : N	FK NOT NULL / FK nula en Horario	RI-18
RP-11	Solicitud la presenta un estudiante y parte de un horario origen; tiene a lo sumo un horario destino, un coordinador de apoyo y un coordinador que la resuelve.	N : 1 / N : 0..1	FK NOT NULL / FK nula	RI-19
RP-12	Unos parámetros de generación pertenecen a exactamente un horario: no se comparten entre horarios.	1 : 1	UNIQUE (idParametro) en Horario	RI-R6 · Supuesto

Sobre RP-12: si dos horarios compartieran la misma fila de parámetros, modificarla cambiaría los dos a la vez, y RI-R6 exige que cada horario conserve los parámetros con que se generó. Dar a cada horario su propia fila, además de hacerla inmutable (RX-03), elimina ese riesgo.

8. Restricciones de especialización (RE)

Código	Restricción	Mecanismo	Origen
RE-01	Cada fila de un subtipo (Administrador, Estudiante, Coordinador, JefeDepartamento) corresponde a un Usuario existente y tiene su mismo identificador.	PK = FK a Usuario (RR-01)	RI-01
RE-02	Especialización total: todo usuario pertenece al menos a un subtipo.	Trigger diferido sobre Usuario	RI-01, RF
RE-03	Especialización solapada: un usuario puede tener varios roles a la vez (por ejemplo, coordinador y jefe de departamento), porque los RF dicen “uno o más roles”.	Sin restricción de exclusión	RF
RE-04	Excepción al solapamiento: un estudiante no puede ser también coordinador, jefe ni administrador.	Trigger en los subtipos	Supuesto

9. Restricciones semánticas (RS)

Son las reglas de negocio que involucran varias tuplas o varias relaciones. RS-01 a RS-08 formalizan las reglas de información RI-R1 a RI-R8; el resto se deducen de los requerimientos funcionales.

Código	Restricción y formalización	Mecanismo	Origen
RS-01	No hay bloques duplicados. $\forall b1, b2 \in \text{BloqueContenido}: b1 \neq b2 \Rightarrow (b1.\text{idAsignatura}, b1.\text{tipoActividad}, \text{norm}(b1.\text{descripcion})) \neq (b2.\text{idAsignatura}, b2.\text{tipoActividad}, \text{norm}(b2.\text{descripcion}))$	Índice único sobre una expresión (RC-08). El intento rechazado se registra en la bitácora de auditoría (RF).	RI-R1
RS-02	Un estudiante no tiene dos horarios activos del mismo programa en el mismo período. $\forall a1, a2 \in \text{AsignacionHorario}: a1 \neq a2 \wedge a1.\text{idEstudiante} = a2.\text{idEstudiante} \wedge a1.\text{estado} = a2.\text{estado} = \text{ACTIVA} \Rightarrow (a1.\text{idPrograma} \neq a2.\text{idPrograma} \vee a1.\text{idPeriodo} \neq a2.\text{idPeriodo})$	Índice único parcial (<code>idEstudiante, idPrograma, idPeriodo</code>) WHERE <code>estado = 'ACTIVA'</code> y FK compuesta a Horario (13.2).	RI-R2 (corregida, ver 14)
RS-03	Solo genera quien tiene <i>puedeGenerar</i> y solo valida quien tiene <i>puedeValidar</i> en ese programa. $\forall h \in \text{Horario} \exists cp \in \text{CoordinadorPrograma}: cp.\text{idUsuario} = h.\text{idCoordinador} \wedge cp.\text{idPrograma} = h.\text{idPrograma} \wedge cp.\text{puedeGenerar}$ $\forall v \in \text{Validacion} \exists cp: (cp.\text{idUsuario}, cp.\text{idPrograma}) = (v.\text{idCoordinador}, v.\text{idPrograma}) \wedge cp.\text{puedeValidar}$	FK compuesta (RR-08, RR-16) y trigger que comprueba el permiso al insertar (13.3). Retirar el permiso después no invalida horarios ya generados.	RI-R3
RS-04	Los bloques de un horario pertenecen al programa del horario. $\forall hb \in \text{HorarioBloque}, h \in \text{Horario}: hb.\text{idHorario} = h.\text{idHorario} \Rightarrow \exists bp \in \text{BloquePrograma}: bp.\text{idBloque} = hb.\text{idBloque} \wedge bp.\text{idPrograma} = h.\text{idPrograma}$	Dos FK compuestas desde HorarioBloque (RR-11). No hace falta trigger.	RI-R4
RS-05	La validación de un horario la hace un coordinador autorizado en el mismo programa del horario, y que no sea quien lo generó. $\forall v \in \text{Validacion}, h \in \text{Horario}: v.\text{idHorario} = h.\text{idHorario} \Rightarrow v.\text{idPrograma} = h.\text{idPrograma} \wedge v.\text{idCoordinador} \neq h.\text{idCoordinador}$	FK compuesta (<code>idHorario, idPrograma</code>) → Horario y trigger para la segunda condición.	RI-17 · Supuesto (≠ generador)
RS-06	El horario conserva los parámetros con que se generó. Ver RX-03 y RP-12.	Trigger que impide UPDATE y DELETE de parámetros ya referenciados.	RI-R6
RS-07	Dos sesiones no ocupan la misma aula en intervalos que se solapan (no basta con que no coincidan exactamente). $\forall s1, s2 \in \text{Sesion}: s1 \neq s2 \wedge s1.\text{idAula} = s2.\text{idAula} \wedge s1.\text{fecha} = s2.\text{fecha} \wedge s1.\text{estado} \neq \text{CANCELADA} \wedge s2.\text{estado} \neq \text{CANCELADA} \Rightarrow s1.\text{horaFin} \leq s2.\text{horaInicio} \vee s2.\text{horaFin} \leq s1.\text{horaInicio}$	Restricción de exclusión EXCLUDE USING gist (13.5).	RI-R7 (generalizada)

Código	Restricción y formalización	Mecanismo	Origen
RS-08	La solicitud registra el apoyo pedido y el resultado del cambio manual. Se garantiza con RT-05, RT-06, RT-07, RS-14 y RX-02.	CHECK y triggers.	RI-R8
RS-09	Un estudiante solo recibe horarios de programas en los que está matriculado. $\forall a \in \text{AsignacionHorario} \exists m \in \text{Matricula}: m.\text{idEstudiante} = a.\text{idEstudiante} \wedge m.\text{idPrograma} = a.\text{idPrograma}$	FK compuesta a Matricula (RR-14).	RI-02, RF
RS-10	Solo se asignan a estudiantes horarios en estado APROBADO .	Trigger BEFORE INSERT en AsignacionHorario.	RF
RS-11	La asistencia solo se registra para estudiantes con asignación ACTIVA al horario de la sesión, y solo si la sesión está IMPARTIDA . $\forall as \in \text{Asistencia}, s \in \text{Sesion}: as.\text{idSesion} = s.\text{idSesion} \Rightarrow s.\text{estado} = \text{IMPARTIDA} \wedge \exists a \in \text{AsignacionHorario}: a.\text{idEstudiante} = as.\text{idEstudiante} \wedge a.\text{idHorario} = s.\text{idHorario} \wedge a.\text{estado} = \text{ACTIVA}$	Trigger en Asistencia.	RI-16, RF
RS-12	Un horario con <i>idRevisionOrigen</i> procede de una revisión RECHAZADA de un horario del mismo programa y período .	Trigger en Horario.	RI-18, RF
RS-13	Solo se revisan horarios en estado VALIDADO . Al registrar la revisión, el estado del horario pasa a APROBADO o RECHAZADO según la decisión, en la misma transacción.	Trigger AFTER INSERT en Revision.	RI-17, RI-18 · Supuesto (orden)
RS-14	El coordinador de apoyo y el que resuelve una solicitud deben estar autorizados en el programa del horario de origen.	Trigger en SolicitudCambio.	RI-19 · Supuesto
RS-15	El horario de origen de una solicitud es una asignación ACTIVA del estudiante. El horario destino es del mismo programa y período y está APROBADO.	Trigger en SolicitudCambio.	RI-19, RF
RS-16	La fecha de una sesión cae dentro del período de su horario. $\forall s \in \text{Sesion}, h \in \text{Horario}, p \in \text{PeriodoAcademico}: s.\text{idHorario} = h.\text{idHorario} \wedge h.\text{idPeriodo} = p.\text{idPeriodo} \Rightarrow p.\text{fechaInicio} \leq s.\text{fecha} \leq p.\text{fechaFin}$	Trigger en Sesion.	RI-11, RI-12
RS-17	Solo se programan sesiones en aulas DISPONIBLES. Una sesión de un bloque de LABORATORIO solo se programa en un aula preparada como laboratorio.	Trigger en Sesion. La segunda parte requiere añadir el atributo Aula.tipo .	RI-13 · Supuesto
RS-18	La cantidad de estudiantes con asignación ACTIVA a un horario no supera la capacidad de ningún aula en la que haya sesiones de ese horario.	Trigger en AsignacionHorario y Sesion.	Supuesto

Código	Restricción y formalización	Mecanismo	Origen
RS-19	Un estudiante no tiene, entre sus horarios ACTIVOS, dos sesiones que se solapen en el tiempo.	Trigger en AsignacionHorario. Complementa RS-02 si “doble asignación” incluye el choque de horas.	RF · Supuesto
RS-20	Los períodos académicos no se solapan y hay como mucho uno en estado ACTIVO.	Restricción de exclusión sobre daterange e índice único parcial.	Supuesto
RS-21	En un período CERRADO no se insertan ni modifican asignaciones, sesiones ni asistencias.	Trigger en las tres relaciones.	Supuesto
RS-22	La composición real de un horario (proporción de bloques por tipo) cumple los parámetros con que se generó.	Lo valida la aplicación al generar (RF: “no guardar un horario inválido”) y se verifica en la consulta 5. No se impone en la BD porque requiere una tolerancia que aún no está definida.	RF

10. Restricciones dinámicas o de transición (RX)

Restringen cómo puede cambiar un estado válido de la base de datos, no solo el estado en sí. Se implementan con triggers BEFORE UPDATE que comparan OLD y NEW.

RX-01 · Ciclo de vida del horario

Estado actual	Evento	Estado nuevo	Observación
GENERADO	Validación con decisión VALIDADO	VALIDADO	Pasa a revisión del jefe de departamento.
GENERADO	Validación con decisión NO_VALIDADO	RECHAZADO	Estado final.
VALIDADO	Revisión con decisión APROBADO	APROBADO	Estado final. Es el “horario final” y el único que se asigna a estudiantes.
VALIDADO	Revisión con decisión RECHAZADO	RECHAZADO	Estado final. Se puede generar un horario nuevo vinculado a la revisión.

No se permite ninguna otra transición. Los estados APROBADO y RECHAZADO son finales. Para corregir un horario rechazado se genera uno nuevo vinculado a la revisión (RS-12); el rechazado no se edita.

Resto de transiciones

Código	Restricción	Origen
RX-02	Solicitud de cambio: PENDIENTE → EN_ATENCION → RESUELTA, o directamente PENDIENTE → RESUELTA. RESUELTA es final: <i>resultado</i> , <i>fechaResolucion</i> e <i>idHorarioDestino</i> no cambian después de resolverse. Si el resultado es ACEPTADA, en la misma transacción la asignación de origen pasa a ANULADA y se crea la asignación ACTIVA al horario destino (RS-02 se sigue cumpliendo).	RI-19, RI-R8

Código	Restricción	Origen
RX-03	Inmutabilidad de la generación: los atributos <i>fechaCreacion</i> , <i>idCoordinador</i> , <i>idPrograma</i> , <i>idPeriodo</i> e <i>idParametro</i> de Horario, todas las filas de ParametrosGeneracion ya referenciadas y las filas de HorarioBloque de un horario que ya no está en estado GENERADO no admiten <code>UPDATE</code> ni <code>DELETE</code> .	RI-R6, RF
RX-04	Sesión: PROGRAMADA → IMPARTIDA o PROGRAMADA → CANCELADA. Los dos estados finales no cambian. Una sesión con asistencias registradas no se cancela ni se borra.	RI-12 · Supuesto
RX-05	Período: PLANIFICADO → ACTIVO → CERRADO. No hay vuelta atrás.	RI-11 · Supuesto
RX-06	Asignación: ACTIVA → ANULADA. Una asignación anulada no se reactiva; si hace falta, se crea una nueva.	RI-15 · Supuesto
RX-07	Validación y revisión son registros de hechos: no admiten <code>UPDATE</code> ni <code>DELETE</code> .	RI-17, RI-18
RX-08	Clasificación de bloques: <i>tipoActividad</i> , <i>nivelComplejidad</i> e <i>idAsignatura</i> de un bloque no cambian si ese bloque ya forma parte de algún horario APROBADO. Si cambiaran, se alterarían los reportes históricos (consultas 2, 4 y 6).	Supuesto

11. Restricciones del histórico analítico (RH)

El histórico de ocupación y asistencia (RI-20) vive en ClickHouse, separado del modelo relacional. ClickHouse **no tiene claves foráneas ni claves únicas que se apliquen**, así que su integridad depende del proceso de carga.

Código	Restricción	Mecanismo
RH-01	Cada par (estudiante, sesión) aparece una sola vez en el histórico de asistencia, aunque la carga se repita.	Motor <code>ReplacingMergeTree</code> con clave de ordenamiento <code>(idEstudiante, idSesion)</code> y consultas con <code>FINAL</code> o con deduplicación en la agregación.
RH-02	Solo se cargan sesiones IMPARTIDAS. Un período solo se da por consolidado cuando está CERRADO.	Filtro en la consulta de extracción desde PostgreSQL.
RH-03	Los atributos descriptivos copiados (complejidad, asignatura, edificio, coordinador creador) son los vigentes en la fecha de la sesión.	Se copian en la carga, sin hacer joins en tiempo de consulta. RX-08 evita que cambien después.
RH-04	Conciliación: para cada período cerrado, el número de asistencias en ClickHouse coincide con el de PostgreSQL.	Consulta de verificación al terminar cada carga; si no coinciden, se registra en auditoría.

12. Reglas que *no* son restricciones de integridad

Las siguientes reglas aparecen en los requerimientos, pero dependen de **quién** ejecuta la operación y no del contenido de los datos. Se implementan como **autorización** en la capa de aplicación (políticas de ASP.NET Core) y,

si se quiere reforzar, con *Row-Level Security* de PostgreSQL. Ponerlas en esta categoría evita una objeción frecuente en la defensa.

Regla	Descripción	Dónde se implementa
RI-R5	Un coordinador puede ver los horarios de otros coordinadores del mismo programa.	Filtro de consulta por los programas del usuario autenticado en <code>CoordinadorPrograma</code> .
RF	Un coordinador solo clasifica los bloques que él creó.	Política <code>usuarioActual = idCreador</code> en el endpoint de actualización.
RF	Un estudiante solo ve sus propios horarios y asistencias.	Filtro por <code>idUsuario</code> autenticado.

13. Implementación en PostgreSQL

Fragmentos de referencia para el equipo de base de datos. Los nombres de las restricciones incluyen su código, así el mensaje de error identifica qué regla se violó.

13.1 Dominios, tupla y unicidad

```
CREATE TYPE tipo_actividad AS ENUM ('TEORICA','PRACTICA','LABORATORIO');
CREATE TYPE nivel_complejidad AS ENUM ('BAJO','MEDIO','ALTO');

CREATE TABLE parametros_generacion (
  id_parametro int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  proporcion_teorica smallint NOT NULL CHECK (proporcion_teorica BETWEEN 0 AND 100),
  proporcion_practica smallint NOT NULL CHECK (proporcion_practica BETWEEN 0 AND 100),
  proporcion_laboratorio smallint NOT NULL CHECK (proporcion_laboratorio BETWEEN 0 AND 100),
  nivel_complejidad nivel_complejidad NOT NULL,
  CONSTRAINT rt01_proporciones_suman_100
    CHECK (proporcion_teorica + proporcion_practica + proporcion_laboratorio = 100)
);

-- RC-08 / RS-01: no hay bloques duplicados
CREATE UNIQUE INDEX rs01_bloque_no_duplicado ON bloque_contenido (
  id_asignatura, tipo_actividad,
  lower(regexp_replace(btrim(descripcion), '\s+', ' ', 'g'))
);
```

Para que dos descripciones que solo se diferencian en las tildes cuenten como duplicadas, se puede usar la extensión `unaccent`. Pero no se puede usar directamente en un índice, porque no está declarada como `IMMUTABLE`: hay que envolverla en una función propia `IMMUTABLE`.

13.2 Claves foráneas compuestas (RS-02, RS-04, RS-09)

```
ALTER TABLE horario
  ADD CONSTRAINT rc10_horario_programa          UNIQUE (id_horario, id_programa),
  ADD CONSTRAINT rc10_horario_programa_periodo UNIQUE (id_horario, id_programa, id_periodo);

CREATE TABLE horario_bloque (
  id_horario int NOT NULL,
  id_programa int NOT NULL,
  id_bloque int NOT NULL,
  PRIMARY KEY (id_horario, id_bloque),
  CONSTRAINT rs04_horario FOREIGN KEY (id_horario, id_programa)
    REFERENCES horario (id_horario, id_programa) ON DELETE CASCADE,
  CONSTRAINT rs04_bloque_del_programa FOREIGN KEY (id_bloque, id_programa)
    REFERENCES bloque_programa (id_bloque, id_programa)
);

CREATE TABLE asignacion_horario (
  id_estudiante int NOT NULL,
  id_horario int NOT NULL,
  id_programa int NOT NULL,
  id_periodo int NOT NULL,
  estado varchar(10) NOT NULL DEFAULT 'ACTIVA'
    CHECK (estado IN ('ACTIVA', 'ANULADA')),
  fecha_asignacion timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (id_estudiante, id_horario),
  CONSTRAINT rr14_horario FOREIGN KEY (id_horario, id_programa, id_periodo)
    REFERENCES horario (id_horario, id_programa, id_periodo),
  CONSTRAINT rs09_matriculado FOREIGN KEY (id_estudiante, id_programa)
    REFERENCES matricula (id_estudiante, id_programa)
);

-- RS-02: un solo horario ACTIVO por estudiante, programa y período
CREATE UNIQUE INDEX rs02_una_asignacion_activa
  ON asignacion_horario (id_estudiante, id_programa, id_periodo)
  WHERE estado = 'ACTIVA';
```

13.3 Trigger de permiso (RS-03)

```
CREATE FUNCTION fn_rs03_puede_generar() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM coordinador_programa cp
    WHERE cp.id_usuario = NEW.id_coordinador
      AND cp.id_programa = NEW.id_programa
      AND cp.puede_generar) THEN
    RAISE EXCEPTION 'RS-03: el coordinador % no puede generar horarios del programa %',
      NEW.id_coordinador, NEW.id_programa
    USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER tg_rs03_puede_generar
  BEFORE INSERT ON horario
  FOR EACH ROW EXECUTE FUNCTION fn_rs03_puede_generar();
```

El trigger se dispara solo en **INSERT**: el permiso se exige en el momento de generar. Los atributos que comprueba no pueden cambiar después (RX-03).

13.4 Participación total con trigger diferido (RP-07)

```
CREATE FUNCTION fn_rp07_horario_con_bloques() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE v_id int := CASE TG_TABLE_NAME WHEN 'horario' THEN NEW.id_horario
                                ELSE OLD.id_horario END;
BEGIN
    IF EXISTS (SELECT 1 FROM horario WHERE id_horario = v_id)
        AND NOT EXISTS (SELECT 1 FROM horario_bloque WHERE id_horario = v_id) THEN
        RAISE EXCEPTION 'RP-07: el horario % debe tener al menos un bloque', v_id;
    END IF;
    RETURN NULL;
END $$;

CREATE CONSTRAINT TRIGGER tg_rp07_alta AFTER INSERT ON horario
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_rp07_horario_con_bloques();

CREATE CONSTRAINT TRIGGER tg_rp07_baja AFTER DELETE ON horario_bloque
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_rp07_horario_con_bloques();
```

El mismo patrón se aplica a RP-02, RP-03, RP-05 y RE-02. El trigger es diferido porque, dentro de la transacción, primero se inserta el horario y después sus bloques: la comprobación debe hacerse al `COMMIT`. En EF Core, ambas inserciones van en el mismo `SaveChanges`, que se ejecuta en una sola transacción.

13.5 Exclusión temporal de aulas (RS-07)

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE sesion
ADD CONSTRAINT rt03_horas CHECK (hora_inicio < hora_fin),
ADD CONSTRAINT rs07_aula_sin_solapamiento EXCLUDE USING gist (
    id_aula WITH =,
    tsrange(fecha + hora_inicio, fecha + hora_fin, '[]') WITH &&
) WHERE (estado <> 'CANCELADA');
```

El intervalo semiabierto `[]` permite sesiones consecutivas (una termina a las 10:00 y la siguiente empieza a las 10:00) sin que se consideren solapadas. La misma técnica, con `daterange`, resuelve RS-20.

13.6 Transición de estados (RX-01)

```
CREATE FUNCTION fn_rx01_transicion_horario() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.estado IS DISTINCT FROM OLD.estado AND NOT (
        (OLD.estado = 'GENERADO' AND NEW.estado IN ('VALIDADO', 'RECHAZADO')) OR
        (OLD.estado = 'VALIDADO' AND NEW.estado IN ('APROBADO', 'RECHAZADO'))
    ) THEN
        RAISE EXCEPTION 'RX-01: transición de horario % -> % no permitida',
            OLD.estado, NEW.estado USING ERRCODE = 'check_violation';
    END IF;
    IF (NEW.fecha_creacion, NEW.id_coordinador, NEW.id_programa, NEW.id_periodo, NEW.id_parametro)
        IS DISTINCT FROM
        (OLD.fecha_creacion, OLD.id_coordinador, OLD.id_programa, OLD.id_periodo, OLD.id_parametro)
    THEN
        RAISE EXCEPTION 'RX-03: los datos de generación del horario % son inmutables',
            OLD.id_horario;
    END IF;
    RETURN NEW;
END $$;

CREATE TRIGGER tg_rx01_horario BEFORE UPDATE ON horario
FOR EACH ROW EXECUTE FUNCTION fn_rx01_transicion_horario();
```

13.7 Resumen de mecanismos

Mecanismo	Restricciones
Tipo / CHECK de columna	RD-01 ... RD-23
CHECK de tabla	RT-01 ... RT-07, RT-09, RT-09
PRIMARY KEY / UNIQUE	RC-01 ... RC-14, RP-12
Índice único parcial o sobre expresión	RC-08 (RS-01), RC-15, RS-02, RS-20
FOREIGN KEY simple y compuesta	RR-01 ... RR-19, RS-04, RS-09, RP-01, RP-04, RP-06, RP-08 ... RP-11
EXCLUDE USING gist	RS-07, RS-20
Trigger BEFORE fila	RS-03, RS-05, RS-10 ... RS-12, RS-14 ... RS-19, RS-21, RE-04, RX-01 ... RX-08
Trigger de restricción diferido	RP-02, RP-03, RP-05, RP-07, RE-02
Lógica de aplicación / autorización	RS-22, sección 12
Proceso de carga (ClickHouse)	RH-01 ... RH-04

14. Inconsistencias detectadas y decisiones pendientes

Al derivar las restricciones aparecieron problemas en los documentos anteriores. Conviene corregirlos antes de que el equipo de base de datos implemente el esquema.

#	Problema	Propuesta
1	RI-R2, tal como está redactada, impide que un estudiante reciba el horario del semestre siguiente del mismo programa, y también los cambios de horario (la asignación de origen sigue existiendo).	Aplicarla solo a asignaciones ACTIVAS y por período (RS-02).
2	RI-09 (Parámetros) no tiene cobertura de asignaturas ni cantidad total de bloques, que el enunciado exige guardar con el horario y que usan las consultas 1 y 5. Además, no está claro qué significa nivelComplejidad como parámetro.	Añadir coberturaAsignaturas (porcentaje de las asignaturas del programa) y cantidadTotalBloques (> 0), y definir o eliminar nivelComplejidad.
3	RI-R7 dice “mismo día y horario”. Si se interpreta como igualdad exacta, una sesión de 10:00 a 12:00 y otra de 11:00 a 13:00 en la misma aula se considerarían válidas.	Formular la regla como solapamiento de intervalos (RS-07).
4	Los RF piden gestionar franjas horarias y una bitácora de auditoría, pero ninguna de las dos está en los requerimientos informacionales.	Añadir las entidades FranjaHoraria y Auditoria en RI, o quitar esos RF.
5	El enunciado habla del “coordinador asignado” al estudiante (el apoyo es “adicional al que tienen asignado”), pero ningún RI lo modela.	Decidir si es el coordinador que generó el horario de origen o una relación explícita Estudiante–Coordinador.
6	Usuario (RI-01) no tiene credenciales, pero los RF exigen autenticación.	Añadir correo (único) y hashContraseña, más activo para la baja lógica.
7	RI-R5 y “solo clasifica sus bloques” aparecen como reglas de información, pero son reglas de autorización.	Trasladarlas a la sección de seguridad (ver sección 12).
8	Está pendiente definir qué hace que dos bloques sean duplicados (RS-01), y si “doble asignación” incluye el choque de horas (RS-19).	Confirmarlo con los profesores.
9	Las restricciones marcadas como Supuesto (rangos de edad, valores de estados, orden validación → revisión, capacidad de aula) son decisiones del equipo.	Revisarlas en reunión y registrar la decisión.